line output in *call_wc.c*, if *execlp* succeeds.

When end-users run C programs from a Linux shell, the shell creates the argument array and invokes the loader—just as we have done earlier. In general, user-level commands that are run in a shell are classified into three types: external commands, internal commands and scripts. You can use the Bash command type to find what type a particular command is—*type ls* will show you that it is external, and *type cd* is a shell built-in. When the shell encounters an external command (like *ls* or *wc*), it calls *fork()* to create a copy of itself; and in the child process, it calls *execlp* with the external command (*wc*) as the argument, like we have seen above. (Internal commands should be implemented as function calls in the shell. For shell scripts, the loader checks the #! first line of the script and calls an appropriate interpreter.) Thus, even as ordinary users, we implicitly use *fork* and *exec* calls everyday!

Please note that the original system call that loads a program is *execve*. Its prototype is:

```
int execve (const char *filename, char *const argv [], char *const
envp []);
```

There are a number of library functions (*execl*, *execlp*, *execle*, *execv* and *execvp*) built on top of it. I have used *execlp* to simplify our discussion. In these function names, *l* means list, and *p*, path. For functions with *l*, you can provide arguments in the form of separate character pointers (as used above); for functions with *v*, arguments are an array of character pointers. Obviously, the second is more flexible, since normally you do not know how many arguments the user will enter. The letter *p* in the name means the function is expected to find the executable by searching a standard path list, normally the *PATH* environment variable, which has paths separated by a colon.

You can even change the environment variables available to the loaded program by using the *envp* parameter of the *exec* calls. Please refer to the manual page for more details.

More concepts: Zombies, orphans and *wait*

To quote Wikipedia, "When a process ends, all memory and resources associated with it are deallocated, so they can be used by other processes. However, the process' entry in the process table remains; this entry is needed to let the parent process read the child's exit status. When a (child) process completes execution, but its entry is still in the process table because its parent has not retrieved the child's exit status, the child is called a *zombie or defunct* process. If the parent executes the *wait* system call and collects the exit status of the child, the zombie entry is removed from the process table."

An *orphan* process is one that is still executing, but whose parent has died. These do not become zombies; instead, they are adopted by *init* (process ID 1), which *waits* on them and reads their exit status.

How does a parent process *wait* for children to complete? See the *mywait.c* code, in the downloaded archive. Compile the code with *gcc mywait.c –Wall* (it's always preferable to compile with *–Wall* to enable all warnings). Run it with arguments, like */a.out 10 20* and you will get the output *My child returned 30*—as expected, which is the sum of the two arguments. The parent *waits* for the child to exit; when the child process terminates, the *wait* call returns, and the exit status of the child is stored in *exitstatus*. Later, it is printed using the macro *WEXITSTATUS*. This is because *wait* also updates the variable at the integer pointer (ours is *&exitstatus*) with more information, like whether the child process terminated abnormally, whether a memory image (core file) has been generated, and so on. To extract only the 1-byte return value (so only numbers up to 255 are possible), use the *WEXITSTATUS* macro.

We can check the return status of the last executed command by echoing the shell variable *$?* and this is implemented using *wait*.

The sample code in *myzombie.c* (in the downloaded archive) will create a zombie or defunct process. Compile the code, and run it in the background (*./a.out &*). Then run the *ps* command to see the defunct entry, like the following:

```
bash-3.00$ ./a.out&
[1] 24849
bash-3.00$ Before fork

bash-3.00$ ps
  PID TTY          TIME CMD
17631 pts/1    00:00:00 bash
24849 pts/1    00:00:00 a.out
24850 pts/1    00:00:00 a.out <defunct>
24856 pts/1    00:00:00 ps
bash-3.00$
```

Zombies will be cleared once the parent terminates, so when you write code for a server, you need to add code that removes zombies, since they can slow the system down. (Typically, you need to register a signal handler for the death-of-child signal, and take appropriate action when it happens. You will understand this after reading the signal section below.)

Signal handling

To understand signal handling, let's start with some questions.

Quite often, we use the *kill* command to terminate processes, like, *kill -9 processid*. What is -9, and what does *kill* do internally?

Consider trivial code like that below, which, when compiled and executed, runs in an infinite loop:

```
#include<stdio.h>
int main () {
        while (1);
}
```

Press *CTRL-C*, and notice that the process is terminated.

Look at faulty code like that below. The global variable *p* is initialised to 0, which is an invalid address—and you can try to assign a value 10, to store at this invalid memory address.